

1) Purpose and Output

- Hook Score: long-horizon discovery ranking score in `product_dynamic_score` (0 to 100).
- Trending Score: short-horizon momentum score in `product_trending_score` (0 to 100).
- Compare Score: session-time ranking among selected devices (2 to 4 devices), returned by compare API.

2) Data Signals Used

Hook Score uses: unique product views, comparisons, wishlist adds, launch age (freshness decay).

Trending Score uses: unique product views, comparisons, view velocity over two windows.

Compare Score uses: device specs + selected variant/base price, plus admin config weights and chipset rules.

3) Hook Score (server/utils/hookScore.js)

Window setup:

- days = 7 (default), lookbackDays = 14
- smoothing s = 5
- freshness parameter tau = 365 days (variable name says half-life, but mathematically this is a decay constant)

Raw components per product:

$\text{views_velocity_raw} = \max(0, ((\text{views_recent} + s)/(\text{views_previous} + s)) - 1)$

$\text{compares_velocity_raw} = \max(0, ((\text{compares_recent} + s)/(\text{compares_previous} + s)) - 1)$

Normalization to 0..100 inside the evaluated product cohort:

$\text{norm_x} = 100 * x / \max(x)$

Applied to `views_recent`, `compares_recent`, `wishlist_recent`, `views_velocity_raw`, `compares_velocity_raw`.

Freshness:

$\text{freshness} = \text{clamp}(100 * \exp(-\text{age_days} / \tau), 0, 100)$

Subscores (default weights):

$\text{buyer_intent} = \text{clamp}((0.55 * \text{norm_views} + 0.30 * \text{norm_compares} + 0.15 * \text{norm_wishlist}) / 1.0, 0, 100)$

$\text{trend_velocity} = \text{clamp}((0.70 * \text{norm_views_velocity} + 0.30 * \text{norm_compares_velocity}) / 1.0, 0, 100)$

Final Hook score:

$\text{hook_score} = \text{clamp}((0.50 * \text{buyer_intent} + 0.30 * \text{trend_velocity} + 0.20 * \text{freshness}) / 1.0, 0, 100)$

Worked example (Phone A):

- `views_recent`=1200, `views_previous`=800, `compares_recent`=240, `compares_previous`=160, `wishlist_recent`=300, `age_days`=90
- `views_velocity_raw`=0.496894, `compares_velocity_raw`=0.484848
- `norm_views`=100.00, `norm_compares`=80.00, `norm_wishlist`=100.00
- `norm_views_velocity`=62.11, `norm_compares_velocity`=80.81
- `buyer_intent`=94.00, `trend_velocity`=67.72, `freshness`=78.15

- hook_score=82.95

4) Trending Score (server/utls/trendingScore.js)

Window setup:

- days = 7 (default), previous window = previous 7 days
- smoothing s = 5

Velocity term:

$\text{velocity_raw} = \max(0, ((\text{views_7d} + s)/(\text{views_prev_7d} + s)) - 1)$

Normalization:

- norm_views, norm_compares, norm_velocity are scaled 0..100 using partition max values by product_type.

Final Trending score (default weights):

$\text{trending_score} = (0.40 \cdot \text{norm_views} + 0.40 \cdot \text{norm_compares} + 0.20 \cdot \text{norm_velocity}) / 1.0$

Worked example (Phone A values from above assumptions):

- norm_views=100.00, norm_compares=80.00, norm_velocity=62.11
- trending_score=84.42

5) Compare Score (server/utls/compareScoring.js)

This is computed only for the selected comparison set and selected variants/prices.

5.1 Performance score

- scoreChipset() uses keyword rules first (admin-managed), then heuristic fallbacks for Snapdragon/Dimensity/Apple/Tensor/Exynos.

5.2 Display score

$\text{refresh_rate_score} = \text{round}(20 + ((\text{clamp}(\text{refresh_rate}, 60, 165) - 60) / (165 - 60)) * 40)$

panel_score from panel text: LTPO=40, AMOLED=34, OLED=32, IPS/LCD=24, TFT=16, etc.

$\text{display_score} = \text{clamp}(\text{refresh_rate_score} + \text{panel_score}, 0, 100)$

5.3 Camera score

$\text{megapixel_score} = \text{clamp}((\text{main_megapixel} / 108) * 65, 18, 65)$

$\text{sensor_score} = \text{clamp}(\text{sensor_count} * 8.75, 10, 35)$

$\text{camera_score} = \text{clamp}(\text{megapixel_score} + \text{sensor_score}, 0, 100)$

5.4 Battery score

Piecewise map by capacity: <=3000:25, <=4000:45, <=4500:60, <=5000:75, <=5500:86, <=6000:94, >6000:100

5.5 Base spec score

$\text{base_spec_score} = 0.40 \cdot \text{performance} + 0.20 \cdot \text{display} + 0.25 \cdot \text{camera} + 0.15 \cdot \text{battery}$

5.6 Price value score

$\text{value_raw} = \text{base_spec_score} / \text{price}$

If no price: value_score=45

If all compared devices have same value_raw: value_score=70

Else: value_score = $35 + ((\text{value_raw} - \text{min_value}) / (\text{max_value} - \text{min_value})) * 65$

5.7 Final compare overall score

overall_score = w_perf*performance + w_disp*display + w_cam*camera + w_batt*battery + w_price*value_score

Default weights: performance=0.36, display=0.20, camera=0.20, battery=0.14, priceValue=0.10

Then rank by overall_score DESC, tie-breaker: lower price first, then name.

Worked compare example (3 phones):

- Phone B: perf=75.00, display=77.00, camera=82.50, battery=86.00, value_score=100.00, overall=80.94, price=30000

- Phone A: perf=95.00, display=77.00, camera=56.34, battery=75.00, value_score=66.17, overall=77.99, price=39999

- Phone C: perf=100.00, display=92.00, camera=56.34, battery=60.00, value_score=35.00, overall=77.57, price=60000

Ranking outcome for this example:

1. Phone B (overall=80.94)

2. Phone A (overall=77.99)

3. Phone C (overall=77.57)

6) API JSON Bodies (final look)

6.1 Recompute Hook score (admin)

POST /api/admin/hook-score/recompute

Body: {} (no payload required)

Sample response:

```
{"ok":true,"updated":123,"results":{"smartphones":{"updated":80},"laptops":{"updated":30},"tvs":{"updated":13}}
```

6.2 Recompute Trending score (admin)

POST /api/admin/trending/recompute

Body: {} (no payload required)

Sample response:

```
{"ok":true,"skipped":false,"updated":250,"days":7}
```

6.3 Manual Trending boost (admin)

POST /api/admin/trending/boost

Body:

```
{"product_id":124,"manual_boost":true,"manual_priority":5,"manual_badge":"editor_pick"}
```

6.4 Update Compare scoring config (admin)

PUT /api/admin/compare-scoring

Body:

```
{"weights":{"performance":36,"display":20,"camera":20,"battery":14,"priceValue":10},"chipset_rules":[{"keyword":"snapdragon"
```

8 gen 3", "score": 95}, {"keyword": "tensor g5", "score": 90}]}

6.5 Get Compare scores for selected products (public)

POST /api/public/compare/scores

Body:

```
{"devices": [{"product_id": 124, "variant_id": 1363}, {"product_id": 69, "variant_index": 0}, {"product_id": 70}]}
```

Sample response:

```
{"scores": [{"product_id": 69, "overall_score": 80.9}, {"product_id": 124, "overall_score": 78.0}, {"product_id": 70, "overall_score": 77.6}]}
```

7) Is there a connection between the three algorithms?

Yes, but only partial:

- Hook Score and Trending Score are behavior-driven and share core engagement signals (views + compares).
- Hook adds wishlist + freshness + a second-level weighted composition.
- Trending is short-horizon momentum and supports manual editorial boosts.
- Compare Score is different: it is spec + price driven for a selected comparison set, not a global popularity rank.

Practical interpretation:

- Hook answers: What should rank well overall in discovery feeds?
- Trending answers: What is hot right now?
- Compare answers: Among selected devices, which is technically and value-wise better?

8) One unified output example (for understanding only)

```
{"product_id": 124, "name": "Phone
```

```
A", "hook_dynamic_score": {"buyer_intent": 94.00, "trend_velocity": 67.72, "freshness": 78.15, "hook_score": 82.95}, "trending": {"views
```